

Response: FlightCore-HAL-Sim: Hardware Abstraction & RTOS Simulator

Dokumentacja Techniczna (Design Document) Wersja: 1.0.0 (Release) **Kategoria:** SITL (Software-In-The-Loop) / Embedded Systems

Ten dokument stanowi kompletną specyfikację architektoniczną i biznesową środowiska symulacyjnego FlightCore. Celem projektu jest implementacja warstwy abstrakcji sprzętu (HAL), mechanizmów komunikacji międzyprocesowej czasu rzeczywistego (Lock-Free IPC) oraz planisty zadań (RTOS Scheduler) w środowisku uruchomieniowym PC, symulującym zachowanie fizycznego mikrokontrolera drona.

1. Wymagania Środowiskowe i Kompilacyjne

Cały projekt musi być bezwzględnie zgodny z poniższymi rygorami:

- **Standard:** C++20.
- **Flagi kompilatora (CMake):** -Wall -Wextra -Werror -pedantic. Odrzucamy każdy kod generujący jakiegokolwiek ostrzeżenia.
- **Brak dynamicznej alokacji w trybie RT:** Zakaz używania new, delete, malloc, free, std::make_shared, std::make_unique oraz kontenerów dynamicznych (np. std::vector) w krytycznych ścieżkach wykonania (przerwania, pętla sterowania). Pamięć musi być alokowana statycznie podczas inicjalizacji.
- **Współbieżność Lock-Free:** Kategoryczny zakaz używania std::mutex, std::lock_guard oraz std::condition_variable w kodzie symulującym sprzęt i przerwania. Komunikacja między wątkami odbywa się wyłącznie za pomocą typów std::atomic z odpowiednio dobranym modelem pamięci (Memory Ordering).

2. Specyfikacja Sprzętowa: IMU-6DOF-FC26 (Memory Map)

System komunikuje się z fizycznym układem inercyjnym poprzez mapowanie pamięci (MMIO). Rozmiar przestrzeni adresowej czujnika to dokładnie 32 bajty. Pamięć zaczyna się od offsetu 0x00. Architektura zakłada Little-Endian.

Offset	Rejestr	Dostęp (SW)	Dostęp (HW)	Opis
0x00	CTRL_REG	R/W	R/O	Rejestr konfiguracyjny (ustawienia sensora).
0x04	STATUS_REG	R/W*	R/W	Rejestr statusowy i flagi

Offset	Rejestr	Dostęp (SW)	Dostęp (HW)	Opis
				przerwań.
0x08	ACCEL_X	R/O	W/O	Surowe dane (Znakowana liczba 32-bit, U2).
0x0C	ACCEL_Y	R/O	W/O	Surowe dane (Znakowana liczba 32-bit, U2).
0x10	ACCEL_Z	R/O	W/O	Surowe dane (Znakowana liczba 32-bit, U2).
0x14	GYRO_X	R/O	W/O	Surowe dane (Znakowana liczba 32-bit, U2).
0x18	GYRO_Y	R/O	W/O	Surowe dane (Znakowana liczba 32-bit, U2).
0x1C	GYRO_Z	R/O	W/O	Surowe dane (Znakowana liczba 32-bit, U2).

(R/W - Read/Write, R/O - Read Only, W/O - Write Only)

2.1. Mapa bitowa rejestru CTRL_REG (0x00)

Wymagany precyzyjny dostęp bitowy. Niezużyte bity muszą mieć wartość 0.

- **Bit 0 (POWER_EN):** 0 = Tryb uśpienia, 1 = Czujnik aktywny.
- **Bity 1-2 (ACCEL_RANGE):** 00 = $\pm 2g$, 01 = $\pm 4g$, 10 = $\pm 8g$, 11 = $\pm 16g$.
- **Bity 3-4 (GYRO_RANGE):** 00 = $\pm 250dps$, 01 = $\pm 500dps$, 10 = $\pm 1000dps$, 11 = $\pm 2000dps$.
- **Bity 5-7 (SAMPLE_RATE):** 000 = 100Hz, 001 = 200Hz, 010 = 400Hz, 011 = 800Hz, 100 = 1000Hz.
- **Bit 8 (INT_EN):** 0 = Wyłączone przerwania sprzętowe, 1 = Sprzęt generuje przerwanie (sygnał wyzwalający ISR) przy nowych danych.
- **Bity 9-31:** Zarezerwowane.

2.2. Mapa bitowa rejestru STATUS_REG (0x04)

- **Bit 0 (DATA_READY):** Sprzęt ustawia na 1, gdy nowe próbki są w rejestrach danych. Oprogramowanie ma obowiązek zapisać w tym bicie 0 natychmiast po odczytaniu danych.
- **Bit 1 (BUFFER_OVERRUN):** Sprzęt ustawia na 1, jeśli nadeszła nowa próbka, a DATA_READY wciąż wynosiło 1 (zgubiona ramka).
- **Bit 2 (HW_ERROR):** Błąd sprzętowy.
- **Bity 3-31:** Zarezerwowane.

3. Kamienie Milowe Projektu (Ticket Tracking)

TICKET #1: Implementacja struktury MMIO dla IMU-6DOF-FC26

Opis Biznesowy: Utworzenie w 100% zgodnej z układem pamięci struktury danych pełniącej rolę mostu między sprzętem a oprogramowaniem. **Wymagania Techniczne:**

1. Zdefiniowanie struktury ImuRegisters odwzorowującej tabelę z sekcji 2.
2. Zastosowanie odpowiednich unii i pól bitowych dla rejestrów konfiguracyjnych.
3. Wykorzystanie metaprogramowania C++20 (static_assert, std::is_standard_layout_v, sizeof) do zagwarantowania w czasie kompilacji rozmiaru równego dokładnie 32 bajty i standardowego układu pamięci.
4. Zamknięcie odpowiednich pól w strukturach atomowych (std::atomic) w celu umożliwienia jednoczesnego dostępu bez użycia std::mutex. Uwaga na ograniczenia kompilatora przy łączeniu atomics i bitfields.
5. Przygotowanie testu w funkcji main: jeden wątek symuluje sprzęt (zapisuje losowe dane, ustawia DATA_READY w pętli), drugi wątek aktywnie sprawdza ten bit i odczytuje wartości. **DoD (Definition of Done):** Kompilacja przechodzi bez błędów. Thread Sanitizer nie zgłasza żadnych wyścigów danych.

TICKET #2: Lock-Free Ring Buffer i Symulacja ISR (Interrupt Service Routine)

Opis Biznesowy: Eliminacja aktywnego oczekiwania (polling) na rzecz modelu opartego na zdarzeniach. Bezpieczne przekazanie danych z kontekstu przerwania do przestrzeni użytkownika. **Wymagania Techniczne:**

1. Stworzenie generycznego (szablonowego) bufora kołowego SpscRingBuffer<T, Size> typu Single-Producer Single-Consumer.
2. Bufor nie może alokować pamięci w czasie działania (tylko w konstruktorze/inicjalizacji).
3. Bufor musi bazować wyłącznie na zmiennych atomowych i odpowiednim Memory Orderingu (Acquire/Release semantics), bez blokowania (lock-free).
4. Przebudowa mechanizmu z Ticketu #1: Wątek sprzętowy (HW), po ustawieniu DATA_READY, wywołuje wstrzykniętą funkcję (tzw. handler ISR). Handler kopiuje 6 wartości z rejestrów DATA_REGS do SpscRingBuffer i czyści flagę DATA_READY. **DoD:** Dwa niezależne wątki potrafią bezbłędnie przesyłać 1 milion struktur z danymi IMU przez bufor bez ani jednego zgubionego lub uszkodzonego pakietu, zweryfikowane przez liczniki kontrolne.

TICKET #3: Tickless Task Scheduler (RTOS Kernel)

Opis Biznesowy: Stworzenie głównego zarządcy czasu, który steruje cyklem życia kontrolera lotu z

twardymi reżimami czasowymi. **Wymagania Techniczne:**

1. Klasa Scheduler zarządzająca statyczną listą zadań (Task). Zadanie ma zdefiniowany interwał wywołania w hercach (np. 400Hz) oraz wskaźnik na funkcję.
2. Scheduler działa w wątku Software. Pobiera aktualny czas systemowy o wysokiej rozdzielczości i wywołuje zadania, których deadline właśnie minął.
3. System usypia wątek (yield/sleep) do czasu następnego najbliższego zadania (architektura Tickless), by nie palić cykli procesora.
4. Rejestrowanie telemetrii opóźnień: Jeśli zadanie trwało zbyt długo i zablokowało kolejne, system zlicza zdarzenie "Deadline Missed". **DoD:** System pozwala na rejestrację 3 testowych zadań (1000Hz, 100Hz, 1Hz) i weryfikuje poprzez wewnętrzny pomiar czasu, że każde uruchamia się z błędem (jitterem) nie większym niż 2 milisekundy na maszynie PC.

TICKET #4: Integracja Control Loop i UDP Telemetry

Opis Biznesowy: Zbudowanie docelowej pętli sprzężenia zwrotnego na podstawie danych z czujnika i wyrzucenie telemetrii na zewnątrz systemu. **Wymagania Techniczne:**

1. Rejestracja zadania ImuTask (np. 500Hz): Wyciąga dane z Ring Buffera i przygotowuje zaktualizowany stan drona.
 2. Rejestracja zadania ControlLoopTask (np. 500Hz): Odpala prosty regulator PID stabilizujący wirtualną wartość (np. symulowany kąt Roll dążący do 0).
 3. Rejestracja zadania TelemetryTask (10Hz): Zbiera aktualny stan i wysyła asynchronicznie przez lokalny socket UDP zadeklarowany pakiet danych. Zapis na socket musi być nieblokujący. **DoD:** Możliwość odbioru pakietów UDP zewnętrznym skryptem (np. w Pythonie), w którym widać zmieniające się surowe wartości IMU oraz wyliczane odpowiedzi regulatora PID bez zakłócania determinizmu czasowego Scheduler'a.
-